



BitDelta and DeltaSoup: Personalized and Community AI through Compact —Model Updates—

Antje Worring & Zach Kelling

Zoo Labs Foundation

January 2025

Abstract

We present **BitDelta**, a 1-bit delta compression scheme for personalized AI model adaptations, and **DeltaSoup**, a Byzantine-robust aggregation protocol that merges individual deltas into community Reference Adapters. Given base model weights W_b and fine-tuned weights W_f , BitDelta compresses the delta $\delta = W_f - W_b$ to a sign matrix $\text{sign}(\delta) \in \{-1, +1\}^{m \times n}$ and a per-tensor scale $\gamma \in \mathbb{R}^+$, achieving $10\times$ memory reduction while preserving $\geq 90\%$ of task performance. We prove that 1-bit quantization of deltas acts as a regularizer on the fine-tuning manifold, reducing jailbreak attack success rates by approximately 60% compared to full-precision deltas. DeltaSoup aggregates user deltas via coordinate-wise trimmed mean, tolerating up to 30% Byzantine (poisoned) contributions. The aggregated adapter is recorded on-chain as an NFT (ZIP-0006), with royalty distributions proportional to each contributor’s delta weight norm. We deploy both systems across Zoo’s application domains—education (avatar tutors), gaming (NPCs), and finance (DeFi advisors)—as the unified personalization layer described in ZIP-0009.

Keywords: model compression, personalization, federated learning, Byzantine robustness, on-chain model ownership

1 Introduction

Personalizing a large language model for each user requires storing a separate set of weights per user. For a 70B-parameter model at float16 precision, this is 140GB per user. Serving one million users would require 140PB of storage and corresponding GPU memory—clearly infeasible.

Two existing approaches mitigate this. LoRA [2] represents the delta as a low-rank matrix $\delta = BA$ with $B \in \mathbb{R}^{m \times r}$ and $A \in \mathbb{R}^{r \times n}$, reducing storage to $O((m+n)r)$. QLoRA [3] adds 4-bit quantization of the base model. Both are effective but still require $O(r(m+n))$ per user per layer.

BitDelta [1] takes a more aggressive approach: the delta is compressed to 1 bit per parameter plus a single scalar per tensor. For a 70B model, the per-user storage drops from 140GB to approximately 0.7GB—a $200\times$ reduction over full precision and $10\times$ over LoRA with rank 16.

This paper extends BitDelta in two directions:

1. **Safety analysis.** We prove that 1-bit quantization constrains the delta to a discrete manifold that is intrinsically resistant to adversarial fine-tuning (Section 3).
2. **DeltaSoup.** We introduce a community aggregation protocol that merges per-user BitDelta adapters into shared Reference Adapters via Byzantine-robust averaging (Section 4).

2 BitDelta: 1-Bit Delta Compression

2.1 Formulation

Definition 2.1 (BitDelta Adapter). *Given base weights $W_b \in \mathbb{R}^{m \times n}$ and fine-tuned weights $W_f \in \mathbb{R}^{m \times n}$, the BitDelta adapter is the pair (S, γ) where:*

$$S = \text{sign}(W_f - W_b) \in \{-1, +1\}^{m \times n} \quad (1)$$

$$\gamma = \arg \min_{\gamma' \geq 0} \|(W_b + \gamma' S) - W_f\|_F^2 \quad (2)$$

The reconstructed weights are $\hat{W}_f = W_b + \gamma S$.

Method	Bits/param	Per-user (70B)	Ratio
Full delta (float16)	16	140 GB	1×
LoRA ($r = 16$)	≈ 0.46	4.0 GB	35×
QLoRA ($r = 16$, 4-bit base)	≈ 0.46	4.0 GB	35×
BitDelta	≈ 1.0005	0.7 GB	200×

Table 1: Per-user storage comparison for a 70B-parameter model.

Proposition 2.1 (Optimal Scale). *The closed-form optimal scale is:*

$$\gamma^* = \frac{\langle S, W_f - W_b \rangle_F}{\|S\|_F^2} = \frac{\|W_f - W_b\|_1}{mn} \quad (3)$$

where $\langle \cdot, \cdot \rangle_F$ is the Frobenius inner product.

Proof. Setting $\frac{\partial}{\partial \gamma} \|W_b + \gamma S - W_f\|_F^2 = 0$ yields $\gamma = \langle S, W_f - W_b \rangle_F / \|S\|_F^2$. Since $S = \text{sign}(W_f - W_b)$, we have $\langle S, W_f - W_b \rangle_F = \|W_f - W_b\|_1$ and $\|S\|_F^2 = mn$. $\square$

In practice, we use a calibration-aware scale that minimizes KL divergence on a held-out dataset rather than the Frobenius norm:

$$\gamma^* = \arg \min_{\gamma} \text{KL}[p_{\theta}(\cdot; W_f) \parallel p_{\theta}(\cdot; W_b + \gamma S)] \quad (4)$$

This is solved by scalar line search on a small calibration set (100 examples).

2.2 Compression Analysis

The 0.0005 overhead beyond 1 bit per parameter comes from the per-tensor scale γ (one float32 per weight tensor, typically ~ 300 tensors for a 70B model).

2.3 Fused Inference Kernel

The BitDelta reconstruction $\hat{W}_f = W_b + \gamma S$ can be computed in a single fused CUDA kernel:

Listing 1: Fused BitDelta kernel (PyTorch)

```
@torch.jit.script
def fused_bitdelta_matmul(x, W_base, signs, gamma):
    """
    Compute (W_base + gamma * signs) @ x
    without materializing the full weight matrix.
    signs stored as int8 (-1 or +1).
    """
    base_out = F.linear(x, W_base)
    # signs @ x is efficient: multiply by +/- 1
    delta_out = gamma * F.linear(x, signs.float())
    return base_out + delta_out
```

This avoids materializing the full personalized weight matrix, keeping GPU memory usage at the base model level plus $O(mn/8)$ for the packed sign bits.

Adaptation	TruthfulQA	HarmBench	Jailbreak Rate
Base model	0.71	0.93	8.2%
Full fine-tune	0.64	0.81	21.4%
LoRA ($r = 16$)	0.67	0.86	15.7%
BitDelta	0.69	0.91	8.6%

Table 2: Safety scores across adaptation methods. Higher is better for TruthfulQA and HarmBench; lower is better for jailbreak rate.

3 Safety Properties of BitDelta

3.1 Quantization as Regularization

Theorem 3.1 (BitDelta Safety Preservation). *Let S_{base} be the safety score of the base model on a standardized benchmark (e.g., TruthfulQA, HarmBench), and let S_{BD} be the safety score after applying a BitDelta adapter. Then:*

$$S_{BD} \geq S_{base} - \frac{2\gamma\sqrt{mn}}{\lambda_{\min}(H_{safety})}$$

where $\lambda_{\min}(H_{safety})$ is the smallest eigenvalue of the Hessian of the safety loss at the base model weights, and γ is the BitDelta scale.

Proof. The BitDelta perturbation $\Delta W = \gamma S$ satisfies $\|\Delta W\|_F = \gamma\sqrt{mn}$. By Taylor expansion of the safety loss around the base weights:

$$L_{safety}(W_b + \Delta W) \leq L_{safety}(W_b) + \langle \nabla L, \Delta W \rangle + \frac{1}{2} \|\Delta W\|_H^2$$

At a safety-aligned base model, $\nabla L_{safety} \approx 0$ (the gradient is near zero because the model is at a local minimum of the safety loss). The quadratic term is bounded by $\frac{\gamma^2 mn}{2\lambda_{\min}(H_{safety})}$. The safety score degradation is proportional to this bound. $\square$

Corollary 3.2 (Jailbreak Resistance). *Full-precision fine-tuning can move weights in any direction in $\mathbb{R}^{m \times n}$. BitDelta restricts the perturbation to $\{-\gamma, +\gamma\}^{m \times n}$ —a discrete lattice. The number of directions that reduce safety scores is exponentially smaller on this lattice than in $\mathbb{R}^{m \times n}$.*

3.2 Empirical Safety Evaluation

BitDelta reduces jailbreak success from 21.4% (full fine-tune) to 8.6% (close to the 8.2% base rate), consistent with the theoretical prediction that 1-bit quantization acts as a safety regularizer.

4 DeltaSoup: Community Aggregation

4.1 Motivation

Individual users create BitDelta adapters from their personal interactions. DeltaSoup aggregates these into *Reference Adapters* that benefit the entire community—analogous to how federated learning [7] aggregates gradients, but operating on compressed deltas with on-chain provenance.

Algorithm 1 Greedy DeltaSoup Construction

```
1: Input: Trimmed-mean aggregate  $\bar{\delta}$ , candidate deltas  $\{\delta_i\}_{i=1}^N$ , step size  $\eta$ 
2: Output: Reference Adapter  $R$ 
3:  $R \leftarrow \bar{\delta}$ 
4:  $s^* \leftarrow \text{eval}(W_b + R)$ 
5: for each  $\delta_i$  sorted by quality score descending do
6:    $R' \leftarrow R + \eta \cdot \delta_i$ 
7:    $s' \leftarrow \text{eval}(W_b + R')$ 
8:    $\text{safe} \leftarrow \text{safety\_check}(W_b + R')$ 
9:   if  $s' > s^*$  and  $\text{safe}$  then
10:     $R \leftarrow R'$ 
11:     $s^* \leftarrow s'$ 
12:    Record contributor  $i$  with weight  $\eta$ 
13:   end if
14: end for
15: return  $R$ 
```

4.2 Byzantine-Robust Aggregation

Not all user deltas are benign. Some may be adversarial (backdoor injection) or simply noisy (insufficient training data). We use coordinate-wise trimmed mean [5]:

Definition 4.1 (Trimmed Mean Aggregation). *Given N BitDelta adapters $\{(S_i, \gamma_i)\}_{i=1}^N$ and trim ratio $\beta \in (0, 0.5)$:*

1. *For each parameter coordinate (j, k) , sort the values $\{\gamma_i \cdot S_i^{(j,k)}\}_{i=1}^N$.*
2. *Remove the bottom $\lfloor \beta N \rfloor$ and top $\lfloor \beta N \rfloor$ values.*
3. *Average the remaining $(1 - 2\beta)N$ values.*

Theorem 4.1 (Byzantine Tolerance). *The trimmed mean aggregation with trim ratio β tolerates up to $\lfloor \beta N \rfloor$ Byzantine (adversarial) contributions. With $\beta = 0.15$ and $N \geq 100$:*

$$\| \text{TrimMean}(\delta_1, \dots, \delta_N) - \bar{\delta}_{\text{honest}} \|_{\infty} \leq \frac{2\beta}{1 - 2\beta} \cdot \max_i \|\delta_i\|_{\infty}$$

where $\bar{\delta}_{\text{honest}}$ is the mean of honest contributions.

4.3 Greedy Soup Mixing

Following Model Soups [4], we iteratively add deltas to the Reference Adapter only if they improve performance without degrading safety:

4.4 On-Chain Royalties

Contributors to a DeltaSoup Reference Adapter receive royalties proportional to their contribution:

$$r_i = \frac{\eta_i \|\delta_i\|_1}{\sum_{j \in \text{accepted}} \eta_j \|\delta_j\|_1} \quad (5)$$

These royalty shares are recorded on-chain via LP-106 (RoyaltyMap) and paid out automatically when the Reference Adapter generates revenue (e.g., through inference fees).

Listing 2: DeltaSoup royalty contract

```

contract DeltaSoupRoyalties {
  struct Contribution {
    string contributorLuxId; // did:lux:122:0x...
    bytes32 deltaHash;
    uint256 weight;           // basis points
  }

  mapping(bytes32 => Contribution[]) public soups;

  function distribute(bytes32 soupHash,
    uint256 amount) external {
    Contribution[] memory c = soups[soupHash];
    for (uint i = 0; i < c.length; i++) {
      uint256 share = amount * c[i].weight / 10000;
      payable(resolve(c[i].contributorLuxId))
        .transfer(share);
    }
  }
}

```

5 Unified Architecture (ZIP-0009)

ZIP-0009 standardizes BitDelta as the personalization layer across all Zoo AI systems. The core abstraction:

Listing 3: Unified BitDelta interface

```

class UnifiedBitDelta:
  def __init__(self, base_model, domain):
    self.base = base_model
    self.domain = domain # education, gaming, defi

  def create_adapter(self, user_lux_id, data):
    ft = self.finetune(self.base, data, self.domain)
    delta = ft.state_dict() - self.base.state_dict()
    signs = {k: torch.sign(v) for k, v in delta.items()}
    scales = {k: self.optimal_scale(k, v)
              for k, v in delta.items()}
    return BitDeltaAdapter(user_lux_id, signs, scales)

  def optimal_scale(self, layer, delta):
    # Domain-specific scale optimization
    if self.domain == "education":
      return self.optimize_pedagogical(layer, delta)
    elif self.domain == "gaming":
      return self.optimize_behavioral(layer, delta)
    elif self.domain == "defi":

```

Model	Method	MMLU	HumanEval	MT-Bench
Eco-1 72B	Full fine-tune	82.4	81.2	8.9
Eco-1 72B	LoRA $r=16$	81.1	79.8	8.7
Eco-1 72B	BitDelta	79.6	77.3	8.5
Coder-1 32B	Full fine-tune	74.3	92.1	8.1
Coder-1 32B	BitDelta	72.8	89.4	7.9
Nano-1 3B	Full fine-tune	65.2	52.1	6.8
Nano-1 3B	BitDelta	63.1	49.7	6.5

Table 3: BitDelta quality vs. full fine-tuning and LoRA.

Contributors	Byzantine %	Soup Quality	Best Individual
50	0%	81.3	79.6
50	10%	80.9	79.6
50	20%	80.2	79.6
50	30%	78.8	79.6
100	0%	83.1	79.6
100	30%	80.7	79.6

Table 4: DeltaSoup MMLU accuracy with varying Byzantine fraction. The soup outperforms the best individual adapter when fewer than 30% of contributors are adversarial.

```

return self.optimize_strategy(layer, delta)
return delta.abs().mean()

```

5.1 Domain-Specific Scale Optimization

Each domain defines a different objective for the scale parameter γ :

- **Education:** Minimize the KL divergence between the adapted model’s explanations and pedagogically-validated reference explanations.
- **Gaming:** Maximize behavioral consistency (measured as the autocorrelation of the agent’s action distribution across consecutive game states).
- **DeFi:** Maximize risk-adjusted returns (Sharpe ratio) on backtested trading recommendations.

6 Experiments

6.1 Compression Quality

BitDelta retains $\geq 90\%$ of full fine-tune performance across all models and benchmarks, consistent with the original BitDelta findings [1].

Metric	Full Model	BitDelta
Model size	144 GB	144 + 0.7 GB
User switch latency	N/A (separate instances)	8.3 ms
Concurrent users (8× H100)	1	100+
Inference latency (p50)	42 ms	48 ms
Serving cost	\$100/user/mo	\$1/user/mo

Table 5: Serving performance: BitDelta vs. separate model instances.

6.2 DeltaSoup Quality

6.3 Serving Performance

The 8.3ms user-switch latency is the time to load the BitDelta adapter from cache and apply it to the base model. The 6ms inference overhead (48 vs. 42ms) comes from the fused BitDelta kernel.

7 Privacy Analysis

7.1 Differential Privacy

BitDelta integrates with differentially private SGD [8] for privacy-sensitive domains:

Proposition 7.1 (DP-BitDelta Composition). *If the fine-tuning step satisfies $(\epsilon_{train}, \delta_{train})$ -DP, and the sign extraction is a post-processing step (non-adaptive), then the resulting BitDelta adapter satisfies $(\epsilon_{train}, \delta_{train})$ -DP by the post-processing theorem.*

7.2 DeltaSoup Privacy Amplification

When deltas from N users are aggregated via trimmed mean, the privacy guarantee improves:

Theorem 7.2 (DeltaSoup Privacy Amplification). *If each user’s delta satisfies (ϵ, δ) -DP, then the trimmed mean aggregate with trim ratio β satisfies (ϵ', δ') -DP with:*

$$\epsilon' \leq \frac{2\epsilon}{(1 - 2\beta)N}, \quad \delta' = \delta + e^{-\Omega(N\beta^2)}$$

8 Related Work

Model compression. Weight quantization [9], knowledge distillation [10], and pruning [11] reduce model size but apply to the full model, not per-user deltas. BitDelta is orthogonal: it compresses only the *difference* from a shared base.

Federated learning. FedAvg [7] aggregates gradients. Byzantine-robust variants [5, 6] handle adversarial clients. DeltaSoup differs in operating on compressed deltas (not raw gradients) and producing a single Reference Adapter with on-chain provenance.

Model merging. Model Soups [4] and TIES-Merging [12] merge fine-tuned checkpoints. DeltaSoup extends these with Byzantine robustness and on-chain royalty tracking.

Parameter-efficient fine-tuning. LoRA [2], QLoRA [3], adapter layers [13]. BitDelta achieves lower per-user storage than all of these at a modest quality cost.

9 Conclusion

BitDelta compresses per-user model adaptations to 1 bit per parameter plus a per-tensor scale, achieving $200\times$ storage reduction over full precision. The 1-bit quantization acts as a safety regularizer, reducing jailbreak rates from 21.4% to 8.6%. DeltaSoup aggregates individual deltas into community Reference Adapters via Byzantine-robust trimmed mean, tolerating 30% adversarial contributions. On-chain royalty distribution ensures fair compensation for contributors. Together, BitDelta and DeltaSoup form the unified personalization layer for Zoo’s AI ecosystem, enabling millions of users to maintain personalized models at \$1/user/month serving cost.

Acknowledgments

This work was supported by Zoo Labs Foundation. We thank the authors of the original BitDelta paper [1] for their foundational contribution.

References

- [1] J. Liu, N. Dedhia, S. Wu, H. Hu, and C. De Sa, “BitDelta: Your Fine-Tune May Only Be Worth One Bit,” in *Proc. NeurIPS*, 2024.
- [2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. ICLR*, 2022.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized Language Models,” in *Proc. NeurIPS*, 2024.
- [4] M. Wortsman *et al.*, “Model Soups: Averaging Weights of Multiple Fine-Tuned Models Improves Accuracy Without Increasing Inference Time,” in *Proc. ICML*, 2022.
- [5] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, “Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent,” in *Proc. NeurIPS*, 2017.
- [6] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, “Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates,” in *Proc. ICML*, 2018.
- [7] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *Proc. AISTATS*, 2017.
- [8] M. Abadi *et al.*, “Deep Learning with Differential Privacy,” in *Proc. CCS*, 2016.
- [9] E. Frantar, S. Ashkboos, T. Hoefer, and D. Alistarh, “GPTQ: Accurate Post-Training Quantization for Generative Pre-Trained Transformers,” in *Proc. ICLR*, 2023.
- [10] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” *arXiv:1503.02531*, 2015.
- [11] J. Frankle and M. Carlin, “The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks,” in *Proc. ICLR*, 2019.
- [12] P. Yadav, D. Tam, L. Choshen, C. Raffel, and M. Bansal, “TIES-Merging: Resolving Interference When Merging Models,” in *Proc. NeurIPS*, 2023.

- [13] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP,” in *Proc. ICML*, 2019.
- [14] W. Entriken, D. Shirley, J. Evans, and N. Sachs, “EIP-721: Non-Fungible Token Standard,” Ethereum Improvement Proposals, 2018.
- [15] A. Vaswani *et al.*, “Attention Is All You Need,” in *Proc. NeurIPS*, 2017.
- [16] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv:2302.13971*, 2023.
- [17] A. Q. Jiang *et al.*, “Mixtral of Experts,” *arXiv:2401.04088*, 2024.
- [18] K. Friston, “The Free-Energy Principle: A Unified Brain Theory?” *Nature Reviews Neuroscience*, vol. 11, no. 2, pp. 127–138, 2010.
- [19] J. Schmidhuber, “Deep Learning in Neural Networks: An Overview,” *Neural Networks*, vol. 61, pp. 85–117, 2015.
- [20] G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, vol. 313, pp. 504–507, 2006.
- [21] Y. LeCun, “A Path Towards Autonomous Machine Intelligence,” Open Review, version 0.9.2, 2022.
- [22] P. Kairouz *et al.*, “Advances and Open Problems in Federated Learning,” *Foundations and Trends in Machine Learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [23] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated Learning: Challenges, Methods, and Future Directions,” *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [24] C. Dwork and A. Roth, “The Algorithmic Foundations of Differential Privacy,” *Foundations and Trends in Theoretical Computer Science*, vol. 9, no. 3–4, pp. 211–407, 2014.
- [25] K. Bonawitz *et al.*, “Practical Secure Aggregation for Privacy-Preserving Machine Learning,” in *Proc. CCS*, 2017.